MarketUp Backend — Architecture & Logic Documentation

This document explains what language(s) the backend is built in, the architecture pattern used, and exactly what the backend logic does — for whoever picks this code up next.

---

1. Languages / Tech Stack

Two **complete, independent, interchangeable** implementations are provided. Both expose the identical API contract, so the Flutter app works against either one without any change. Pick whichever matches your team.

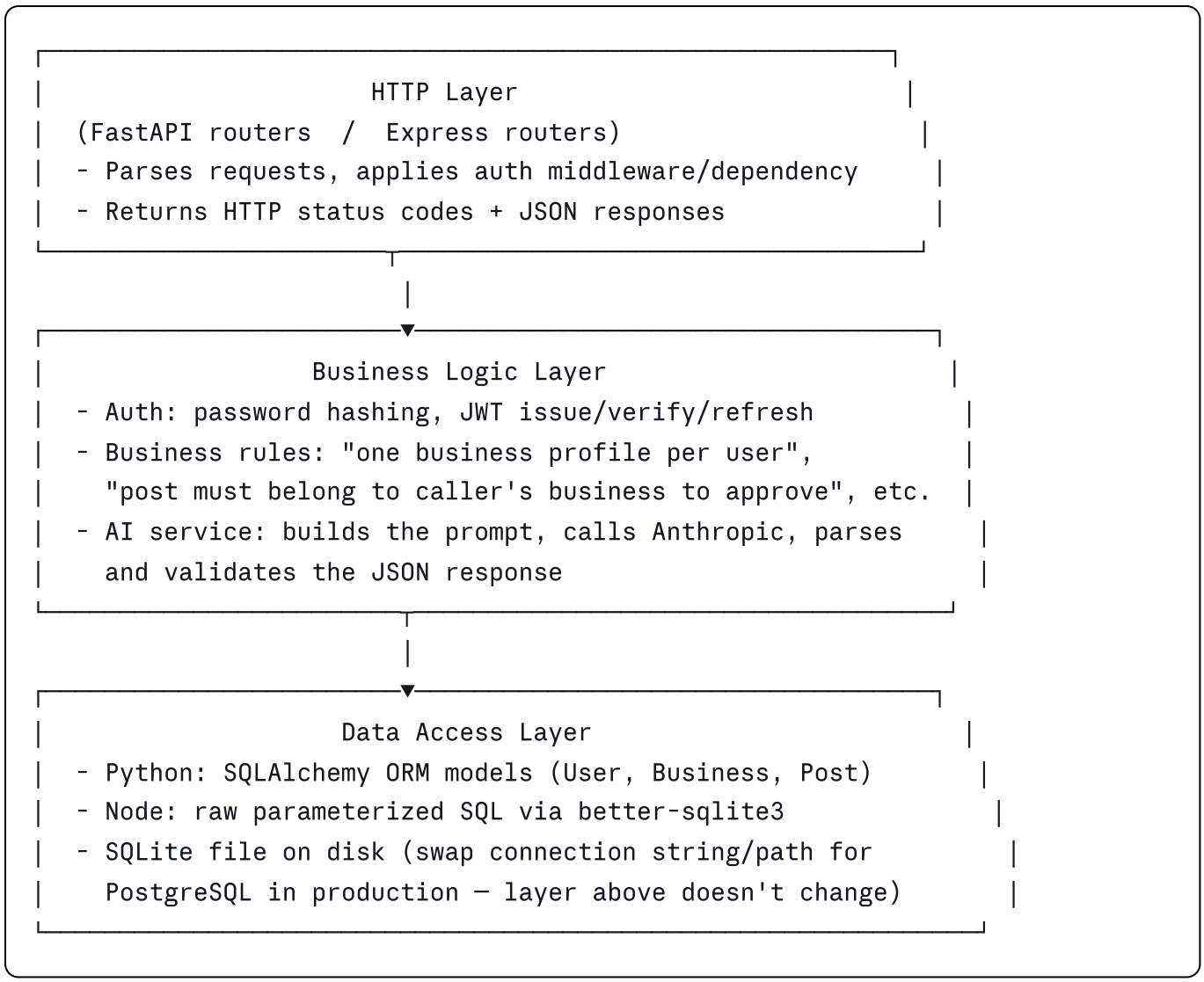
	Python version (`backend/`)	**Node.js version (`backend-js/`)**
Language	Python 3.12	JavaScript (Node.js 22, ES Modules)
Web framework	FastAPI	Express
Database	SQLite (via SQLAlchemy ORM)	SQLite (via better-sqlite3, direct SQL)
Auth	JWT (python-jose) + bcrypt (direct, not passlib)	JWT (jsonwebtoken) + bcryptjs
AI provider SDK	`anthropic` (official Python SDK)	`@anthropic-ai/sdk` (official Node SDK)
Validation	Pydantic schemas	Manual validation in route handlers
Server	Uvicorn (ASGI)	Node's built-in HTTP server via Express

Both were run and tested live during development (not just written and assumed to work) — signup, login, wrong-password rejection, business create/duplicate-rejection, AI-generate error handling, post listing, and auth-guard checks were all exercised against a running instance of each.

---

2. Architecture

Both backends follow the same **layered architecture** — a standard, well-understood pattern for this size of API (roughly: routes → business logic → data access), which keeps things easy to extend (e.g. swapping SQLite for PostgreSQL later touches only the data layer, not the routes).



Cross-cutting concerns, applied the same way in both versions:

- **Auth middleware** sits in front of every protected route, decodes the JWT, loads the user, and rejects with 401/403 before the route handler ever runs — routes never have to re-implement auth checks.
- **CORS** is enabled globally so the Flutter app (or a web client) can call the API from a different origin during development.
- **Errors** are returned as `{"detail": "human-readable message"}` with an appropriate HTTP status code (422 validation, 401 auth, 403 forbidden, 404 not found, 400 conflict, 502/503 upstream AI failure, 500 unexpected) — consistent across both languages so client code doesn't need per-backend error handling.

Folder layout (both versions mirror each other)

```
backend/                          backend-js/
├── app/                          ├── src/
│   ├── main.py                   │   ├── server.js          (entrypoint)
│   ├── core/                     │   ├── config.js
│   │   ├── config.py             │   ├── db.js
│   │   ├── security.py           │   ├── utils/security.js
│   │   └── deps.py               │   ├── middleware/auth.js
```

```
|   ├── db.py                  |
|   ├── models.py              |
|   ├── schemas.py             |
|   ├── routers/                |   ├── routes/
|   |   ├── auth.py             |   |   ├── auth.js
|   |   ├── business.py         |   |   ├── business.js
|   |   └── ai_content.py       |   |   └── aiContent.js
|   └── services/               |   └── services/
|       └── ai_service.py       |       └── aiService.js
```

---

3. Backend Logic — What Actually Happens

3.1 Authentication

1. **Signup** (`POST /auth/signup`): validates email + password (≥8 chars), checks the email isn't already registered, hashes the password with bcrypt, stores the user row, and immediately issues a token pair.
2. **Login** (`POST /auth/login`): looks up the user by email, verifies the password against the stored bcrypt hash, issues a token pair on success or a 401 on failure — the error message is deliberately generic ("Invalid email or password") so it doesn't reveal whether the email exists.
3. **Tokens**: an **access token** (15 min expiry) is sent as `Authorization: Bearer <token>` on every protected request; a **refresh token** (30 day expiry) is used only to get a new access token when the old one expires. Both are signed JWTs containing the user's ID (`sub`) and a `type` claim (`access` or `refresh`) so one can't be used in place of the other.
4. **Auth guard**: every protected route runs through a shared dependency/middleware that decodes the bearer token, confirms it's an access token (not a refresh token), loads the user row from the database, and attaches it to the request — routes just read `req.user` / `current_user` without re-checking anything.

3.2 Business Profile

- **One business per user**, enforced at the database level (unique constraint on `user_id` in `businesses`) and checked explicitly before insert so the API returns a clean `400` instead of a raw DB error.
- `POST /businesses` creates the profile (name, sector, description, address, brand colors, etc.) — this is the data the AI generation step reads from later.
- `GET /businesses/me` / `PATCH /businesses/me` read/update the caller's own profile only — there is no way to fetch or edit another business through this API (every query is scoped by `user_id` from the auth token, never by an ID passed in the request body).

3.3 AI Content Generation — the core feature

`POST /ai/generate` is the heart of the product. Step by step:

1. Look up the caller's business profile (404 if onboarding isn't done yet).
2. Build a **system prompt** that locks the model into a strict behavior: act as MarketUp's social media copywriter for small Indian businesses, and respond with *only* a single JSON object (no markdown fences, no preamble) matching an exact schema: `caption`, `hashtags`, `cta`, `image_prompt`.
3. Build a **user prompt** from the business's actual data — name, sector, description, address, brand colors — plus either the caller-supplied `occasion` (e.g. "Diwali") or an instruction to pick something naturally relevant to today's date if none was given.
4. Call the real Anthropic API (`client.messages.create(...)`) with that system + user prompt.
5. Parse the model's response as JSON (defensively stripping ```` ```json ```` fences if the model adds them despite instructions), and validate the expected keys are present.
6. Persist a new `posts` row with `status = 'pending'` and return it to the client.
7. **Error handling**: if `ANTHROPIC_API_KEY` isn't set, the service throws a specific, catchable error and the route returns `503` with a clear message — this was deliberately tested (see §1) so the failure mode is honest rather than a generic crash.

3.4 Approve / Reject / List

- `GET /businesses/me/posts` lists posts for the caller's business only, optionally filtered by `status`.
- `POST /posts/:id/approve` — checks the post belongs to the caller's business (not just any post ID), then sets `status = 'scheduled'` and stamps `approved_at`. (Actual publishing to Instagram/Facebook/WhatsApp is not implemented yet — see §4.)
- `POST /posts/:id/reject` — same ownership check, sets `status = 'rejected'`.

3.5 Data Model

Three tables (same shape in both SQL layers):

```
users        (id, email, password_hash, status, created_at)
businesses   (id, user_id → users, business_name, owner_name, sector,
              sub_sector, description, address, phone, email,
              brand_colors [JSON], logo_url, website, created_at)
posts        (id, business_id → businesses, occasion, caption,
              hashtags [JSON], cta, image_prompt, status,
              ai_model_used, created_at, approved_at)
```

`brand_colors` and `hashtags` are stored as JSON text in SQLite (both versions), which is fine at this scale and trivially portable to PostgreSQL's native `JSONB` type later without

changing the API shape.

---

4. What's Deliberately Not in the Backend Yet

- **Publishing to Instagram/Facebook/WhatsApp** — needs a live Meta Developer App (client ID/secret, redirect URI) that only you can create.
- **Razorpay subscription billing** — needs a live Razorpay account and a created Plan.

Both are scoped out because no backend code can substitute for credentials that only exist once you've registered with those platforms — the original full product spec (shared earlier) documents the exact integration flow for each (Meta OAuth flow and Razorpay subscription flow sections) so either backend can be extended once you have those accounts.